

ADVANCED COMPUTER ARCHITECTURE

Francesco Leporati

Emanuele Torti

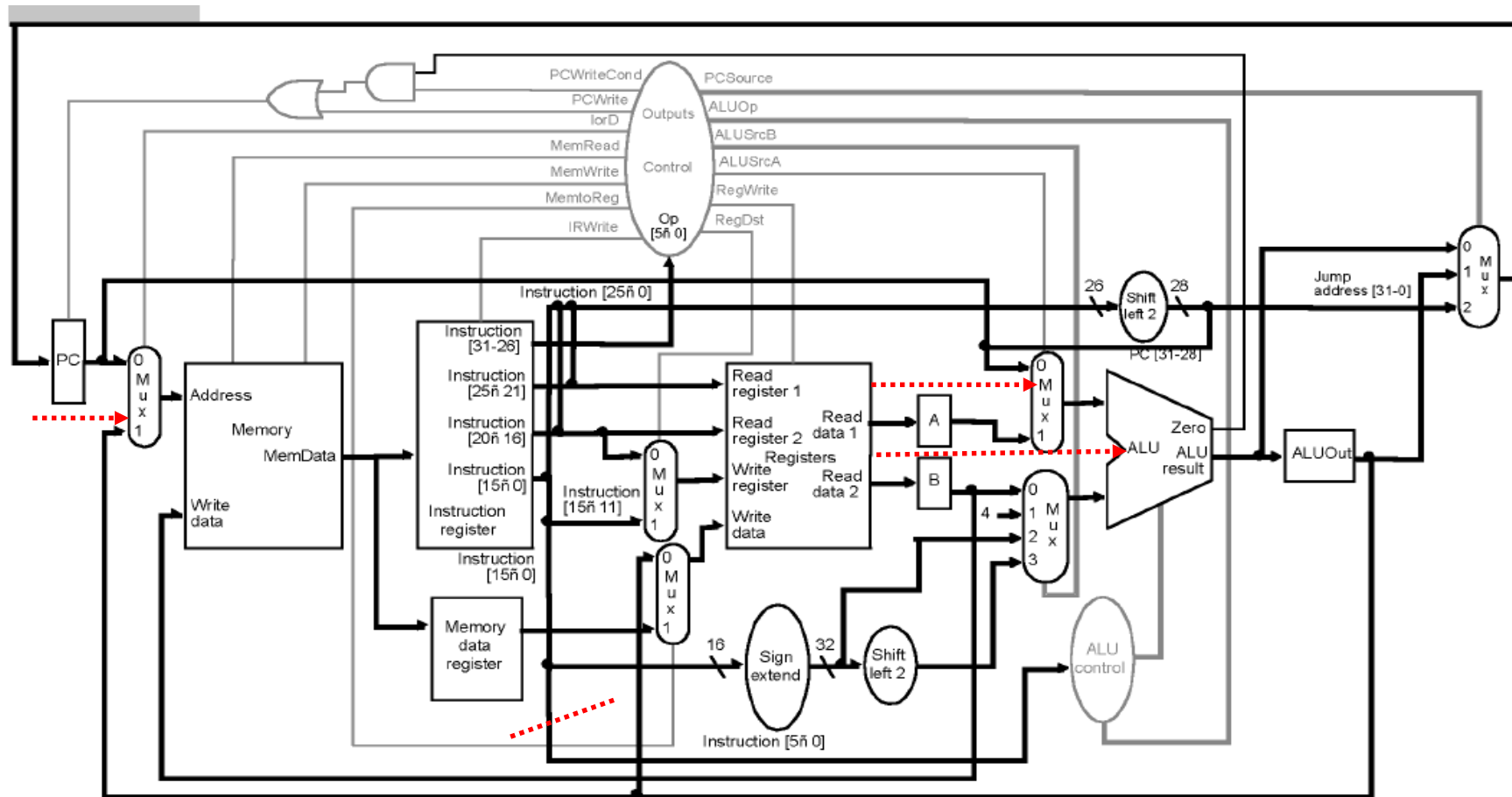
Introduction

- MIPS Processor



- RISC simplified architecture to implement 5 assembly instructions
- Load/Store word, Add, Branch, Jump
- Design and implementation of the Control Unit
- New instructions in the exam $\Rightarrow$ digital design

Target



Register set

Policy of Use Conventions

Name	Register number	Usage
\$zero	0	the constant value 0
\$v0-\$v1	2-3	values for results and expression evaluation
\$a0-\$a3	4-7	arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved
\$t8-\$t9	24-25	more temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	return address

Instruction set

MIPS operands

Name	Example	Comments
32 registers	<code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code>	Fast locations for data. In MIPS, data must be in registers to perform arithmetic. MIPS register \$zero always equals 0. Register \$at is reserved for the assembler to handle large constants.
2^{30} memory words	<code>Memory[0], Memory[4], ..., Memory[4294967292]</code>	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential words differ by 4. Memory holds data structures, such as arrays, and spilled registers, such as those saved on procedure calls.

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	<code>add \$s1, \$s2, \$s3</code>	$\$s1 = \$s2 + \$s3$	Three operands; data in registers
	subtract	<code>sub \$s1, \$s2, \$s3</code>	$\$s1 = \$s2 - \$s3$	Three operands; data in registers
	add immediate	<code>addi \$s1, \$s2, 100</code>	$\$s1 = \$s2 + 100$	Used to add constants
Data transfer	load word	<code>lw \$s1, 100(\$s2)</code>	$\$s1 = \text{Memory}[\$s2 + 100]$	Word from memory to register
	store word	<code>sw \$s1, 100(\$s2)</code>	$\text{Memory}[\$s2 + 100] = \$s1$	Word from register to memory
	load byte	<code>lb \$s1, 100(\$s2)</code>	$\$s1 = \text{Memory}[\$s2 + 100]$	Byte from memory to register
	store byte	<code>sb \$s1, 100(\$s2)</code>	$\text{Memory}[\$s2 + 100] = \$s1$	Byte from register to memory
	load upper immediate	<code>lui \$s1, 100</code>	$\$s1 = 100 * 2^{16}$	Loads constant in upper 16 bits
Conditional branch	branch on equal	<code>beq \$s1, \$s2, 25</code>	if ($\$s1 == \$s2$) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	<code>bne \$s1, \$s2, 25</code>	if ($\$s1 != \$s2$) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	<code>slt \$s1, \$s2, \$s3</code>	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than; for beq, bne
	set less than immediate	<code>slti \$s1, \$s2, 100</code>	if ($\$s2 < 100$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than constant
Unconditional jump	jump	<code>j 2500</code>	go to 10000	Jump to target address
	jump register	<code>jr \$ra</code>	go to \$ra	For switch, procedure return
	jump and link	<code>jal 2500</code>	$\$ra = \text{PC} + 4$; go to 10000	For procedure call

Addressing modes

Register addressing

operando = registro

Base addressing

operando = cost(\$reg)

Immediate addressing

operando = costante

PC relative addressing

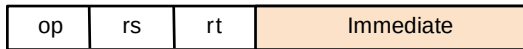
operando = PC+4+costante

Pseudodirect addressing

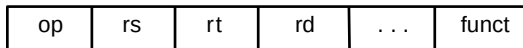
**operando = 4 bit PC & 26 bit
bassi istruzione & “00”**

Addressing modes

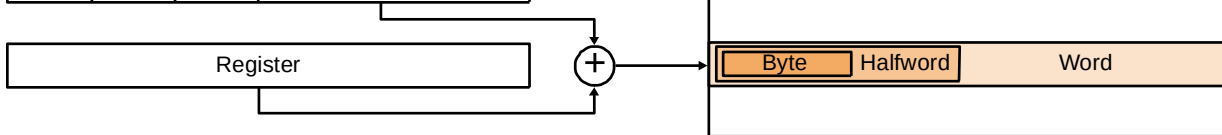
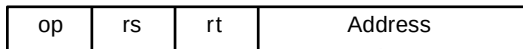
1. Immediate addressing



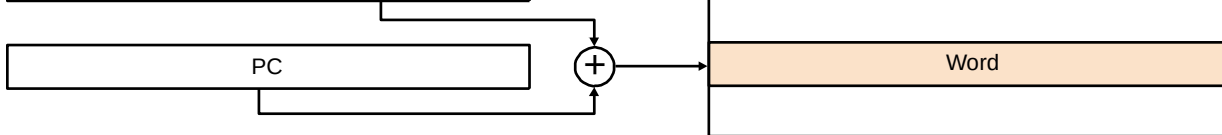
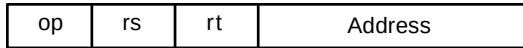
2. Register addressing



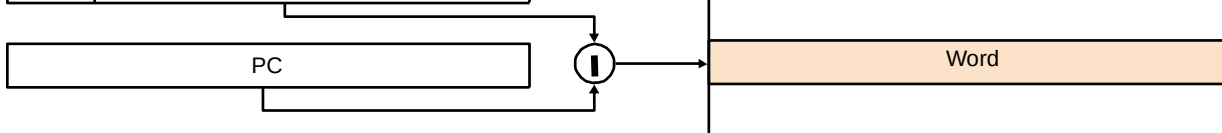
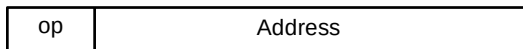
3. Base addressing



4. PC-relative addressing



5. Pseudodirect addressing



Logic operations

- *Shifts:*

shift left log. 8 bit 00000000D → 000000D00 **sll \$t2, \$s0, 8**
shift right log. 4 bit 000000D00 → 0000000D0 **srl \$t2, \$s0, 4**

- *Rformat:*

OP	RS	RT	RD	Shamt	Funct
0	0	16	10	8	0

- *AND bit a bit* \$t1=00003C00 \$t2=00000D00
 AND \$t0, \$t1, \$t2 \$t0=00000C00
- *OR bit a bit* OR \$t0, \$t1, \$t2 \$t0=00003D00

R-format

- Instructions, like registers and words of data, are also 32 bits long
 - Example: `add $t0, $s1, $s2`
 - registers have numbers, `$t0=8`, `$s1=17`, `$s2=18`

- Instruction Format:

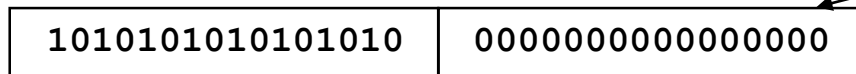
000000	10001	10010	01000	00000	100000
op	rs	rt	rd	shamt	funct

How about larger constants?

- We'd like to be able to load a 32 bit constant into a register
- Must use two instructions, new "load upper immediate" instruction

```
lui $t0, 1010101010101010
```

filled with zeros



1010101010101010	0000000000000000
------------------	------------------

- Then must get the lower order bits right, i.e.,

```
ori $t0, $t0, 1010101010101010
```

ori

1010101010101010	0000000000000000
0000000000000000	1010101010101010

1010101010101010	1010101010101010
------------------	------------------

Immediate format

- Consider the load-word and store-word instructions,
 - What would the regularity principle have us do?
 - New principle: Good design demands a compromise
- Introduce a new type of instruction format
 - I-type for data transfer instructions
 - other format was R-type for register
- Example: `lw $t0, 32($s2)`

35	18	8	32
----	----	---	----

op	rs	rt	16 bit number
----	----	----	---------------

- Where's the compromise?

Addresses in Branches and Jumps

- Instructions:

<code>bne \$t4,\$t5,Label</code>	Next instruction is at Label if $\$t4 \neq \$t5$
<code>beq \$t4,\$t5,Label</code>	Next instruction is at Label if $\$t4 = \$t5$
<code>j Label</code>	Next instruction is at Label

- Formats:

I	op	rs	rt	16 bit address
J	op	26 bit address		

- Addresses are not 32 bits
 - How do we handle this with load and store instructions?

Conditional jumps (Branch) addresssing

loop:	mult	\$9, \$19, \$10	add	\$19, \$19, \$20
	lw	\$8, 1000(\$9)	j	loop
	bne	\$8, \$21, exit	exit:	...

80000	0	19	10	9	0	24
80004	35	9	8		1000	
80008	5	8	21		8 (bytes not word)	
80012	0	19	20	19	0	32
80016	2				80000 (not reale address)	
80020	...					

Absolute jumps addressing

- LOOP 0x00080000

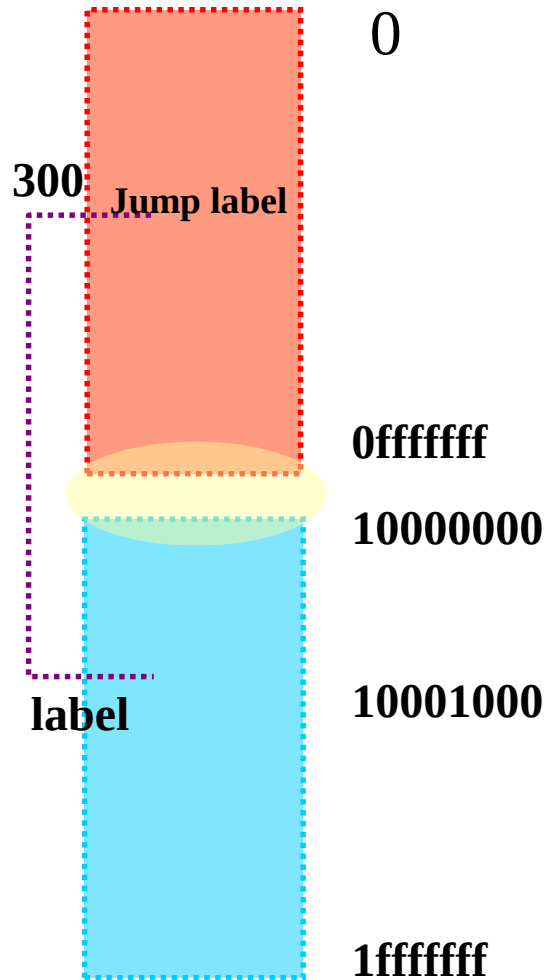
- 0000 0000 0000 1000 0000 0000 0000 0000

- Object code jump

- 000010 0000 0000 1000 0000 0000 0000 00

- 0 8 0 2 0 0 0 0

Issues in absolute jumps



- *label* = 0x10001000
- Object code Jump
 - 0x08000400
- Wrong target address achieved in the jump (erroneo)
 - 0x00001000
- Solution!
 - Jump close to the border of the page (red)
 - Branch to a whatever address in the new page (blu area)
 - Jump from that address to *label*

DELAYED BRANCHES

NOT DELAYED BRANCH

0x16600002  bne \$19, \$0, 8
break \$s0
div \$18, \$19
mflo \$17

"CORRECT" IMPLEMENTATION

0x16600001  bne \$19, \$0, 4
break \$s0
div \$18, \$19
mflo \$17

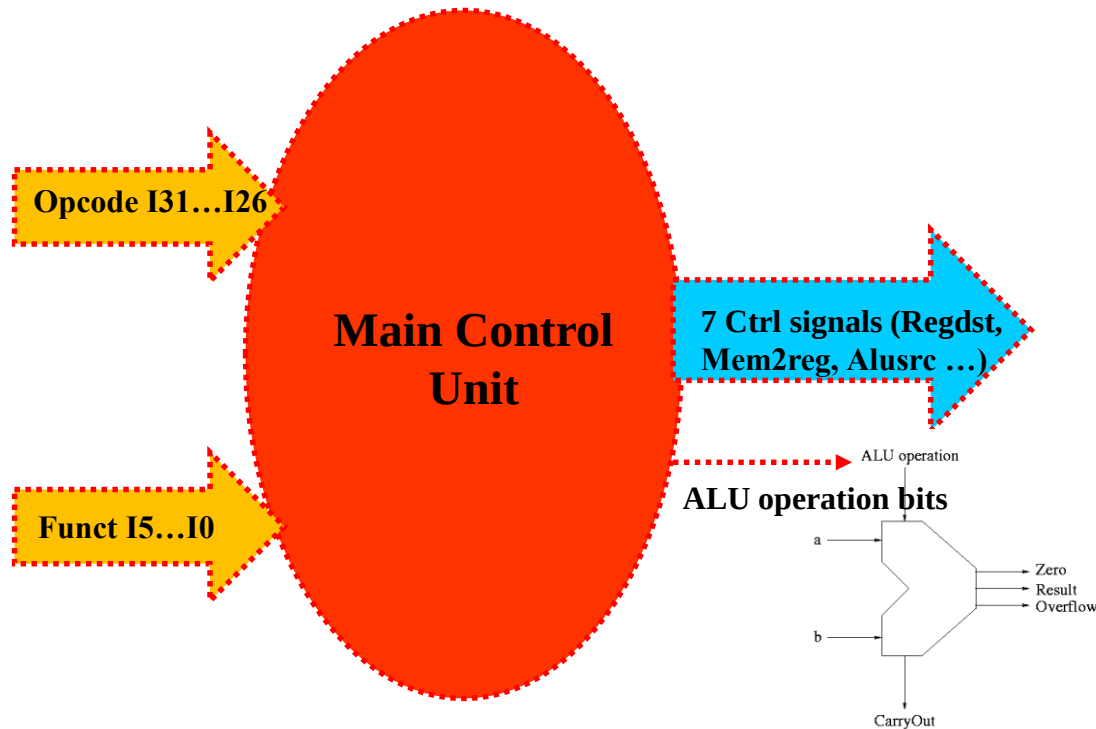
DELAYED BRANCH

0x16600001  bne \$19, \$0, 4
break \$s0
div \$18, \$19
mflo \$17

"CORRECT" IMPLEMENTATION

0x16600000  bne \$19, \$0, 0
break \$s0
div \$18, \$19
mflo \$17

Main Control Unit and ALU Control Unit - 1

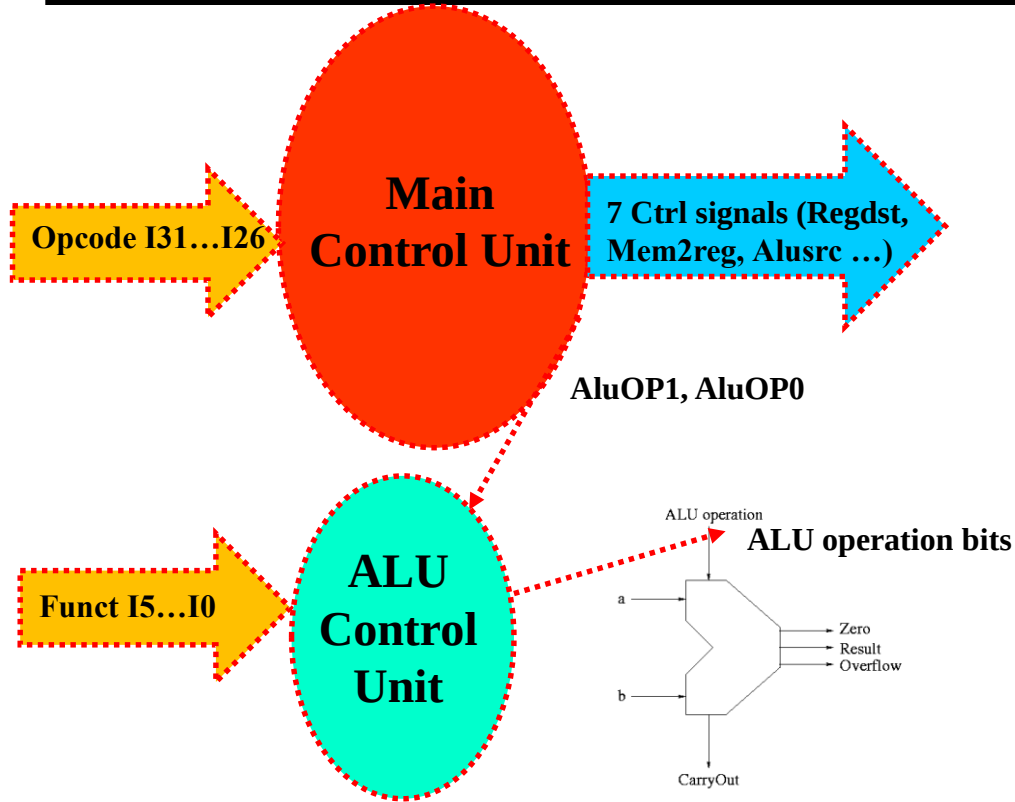


What is the size of a single big Control Unit managing both control signals and the operations of the ALU?

$$2^{12} \times 10 = 40 \text{ Kbit}$$

Can we do something better?

Main Control Unit and ALU Control Unit - 2



The new memory requirements is

$$2^6 \times 9 + 2^8 \times 3 = 1344 \text{ bits}$$

DIVIDE et IMPERA

Main Control Unit and ALU Control Unit - 3

Instruction opcode	ALUOp	Instruction operation	Func field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	010
SW	00	store word	XXXXXX	add	010
Branch equal	01	branch equal	XXXXXX	subtract	110
R-type	10	add	100000	add	010
R-type	10	subtract	100010	subtract	110
R-type	10	AND	100100	and	000
R-type	10	OR	100101	or	001
R-type	10	set on less than	101010	set on less than	111

ALUOp		Func field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	010
X	1	X	X	X	X	X	X	110
1	X	X	X	0	0	0	0	010
1	X	X	X	0	0	1	0	110
1	X	X	X	0	1	0	0	000
1	X	X	X	0	1	0	1	001
1	X	X	X	1	0	1	0	111

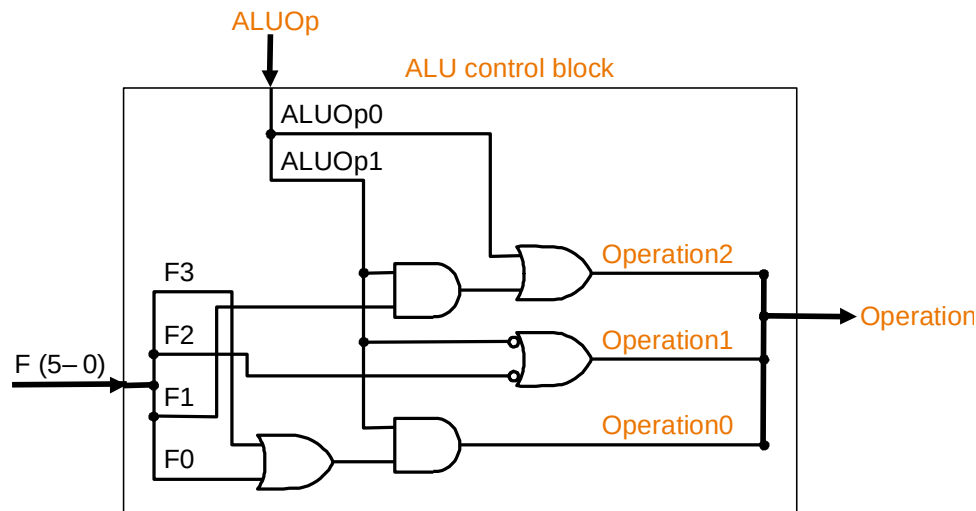
Main Control Unit and ALU Control Unit - 4

ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
x	1	x	x	x	x	x	x	Operation 2
1	x	x	x	x	x	1	x	
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	x	x	x	x	x	x	x	Operation 1
x	x	x	x	x	0	x	x	
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
1	x	x	x	x	x	x	1	Operation 0
1	x	x	x	1	x	x	x	

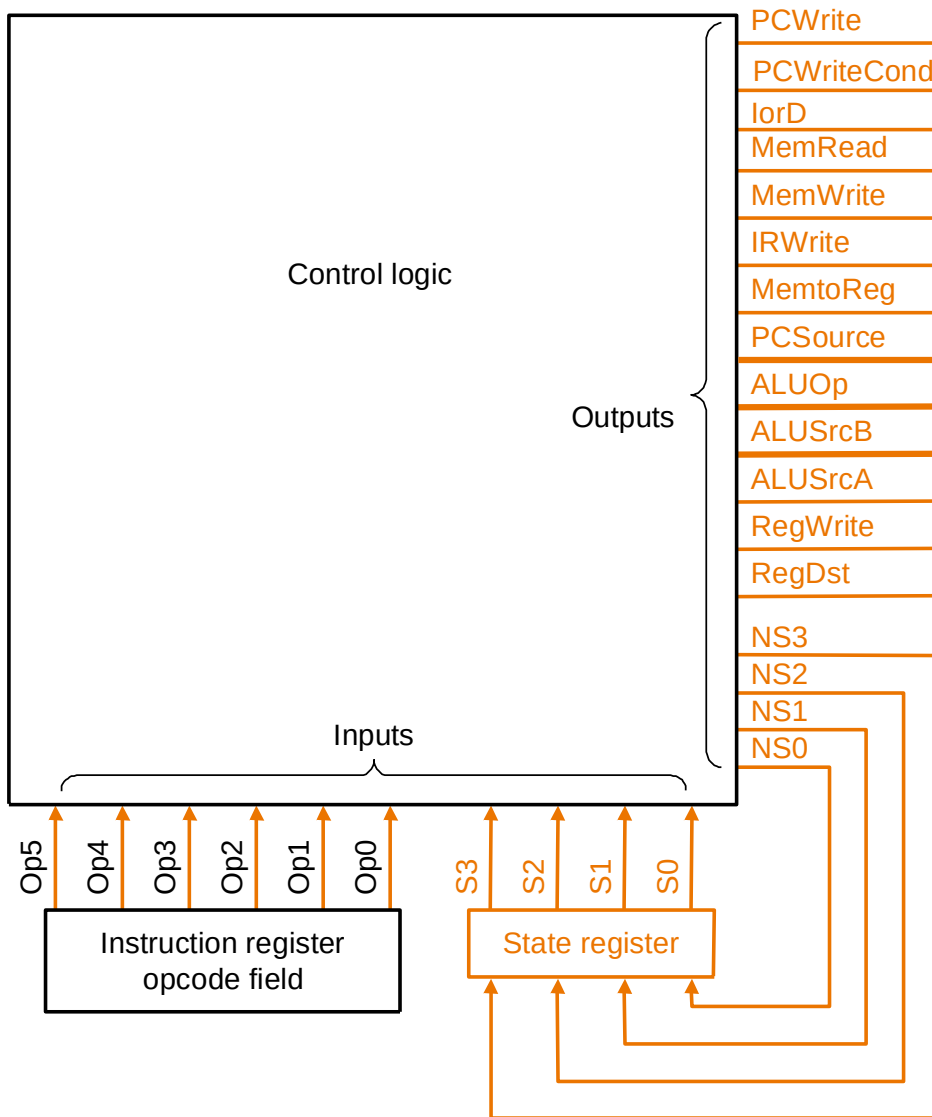
ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	010
X	1	X	X	X	X	X	X	110
1	X	X	X	0	0	0	0	010
1	X	X	X	0	0	1	0	110
1	X	X	X	0	1	0	0	000
1	X	X	X	0	1	0	1	001
1	X	X	X	1	0	1	0	111

Main Control Unit and ALU Control Unit - 5

ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
x	1	x	x	x	x	x	x	Operation 2
1	x	x	x	x	x	1	x	
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	x	x	x	x	x	x	x	Operation 1
x	x	x	x	x	0	x	x	
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
1	x	x	x	x	x	x	1	Operation 0
1	x	x	x	1	x	x	x	



Multi cycle control unit – single ROM implementing the FSM



S3, S2, S1, S0: bits used to encode the current state
NS3, NS2, NS1, NS0: bits used to encode the future state

The control logic implements a truth table relating the control outputs to the future state (see table below)

Logic equations for the control unit

<i>PC Write</i>	<i>PC Write Cond</i>	<i>lorD</i>	<i>Mem Read</i>	<i>Mem write</i>	<i>IR Write</i>	<i>Mem toreg</i>	<i>PC Src 1</i>	<i>PC Src 0</i>	<i>ALU Op1</i>	<i>ALU Op0</i>	<i>ALU SrcB 1</i>	<i>ALU SrcB 0</i>	<i>ALU SrcA</i>	<i>Reg Write</i>	<i>Reg Dst</i>
0+9	8	3+5	0+3	5	0	4	9	8	6	8	1+2	0+1	2+6+8	4+7	7

Control signals depend only on the current states

Fragment of the control logic corresponding to the PCWrite signal active	S3	S2	S1	S0
	0	0	0	0
	1	0	0	1

<i>Next State</i>	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>	<i>8</i>	<i>9</i>
<i>Current State</i>	4+5+7+8+9	0	1	2	3	2	1	6	1	1

Example: table for the bit 0 in the next state (NS0)

NS0=1 in the next states 1, 3, 5, 7, 9 (0001, 0011, 0101, 0111, 1001)

Next st. 1= curr. st. 0 Next st. 3= curr. st. 2 AND lw Next st. curr. st. 2 AND sw

Next st. 7= curr. st. 6 Next st. 9= curr. st. 1 AND jump

NS0 or of all these conditions

Op5	Op4	Op3	Op2	Op1	Op0	S3	S2	S1	S0
x	x	x	x	x	x	0	0	0	0
1	0	0	0	1	1	0	0	1	0
1	0	1	0	1	1	0	0	1	0
x	x	x	x	x	x	0	1	1	0
0	0	0	0	1	0	0	0	0	1

2_ROMs implementation – control signals (ROM1)

Uscite/Ingressi	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
PCWrite	1	0	0	0	0	0	0	0	0	1
PCWrite Cond	0	0	0	0	0	0	0	0	1	0
IorD	0	0	0	1	0	1	0	0	0	0
MemRD	1	0	0	1	0	0	0	0	0	0
MemWR	0	0	0	0	0	1	0	0	0	0
IRWrite	1	0	0	0	0	0	0	0	0	0
Memtoreg	0	0	0	0	1	0	0	0	0	0
PCSrc1	0	0	0	0	0	0	0	0	0	1
PCSrc0	0	0	0	0	0	0	0	0	1	0
AluOp0	0	0	0	0	0	0	1	0	0	0
AluOp1	0	0	0	0	0	0	0	0	1	0
AlusrcB1	0	1	1	0	0	0	0	0	0	0
AlusrcB0	1	1	0	0	0	0	0	0	0	0
AlusrcA	0	0	1	0	0	0	1	0	1	0
RegWr	0	0	0	0	1	0	0	1	0	0
RegDst	0	0	0	0	0	0	0	1	0	0

20 Kbit ROM = $2^{10} \times 20$: 10 inputs (6 opcodes + 4 current status bits), 20 outputs (16 control lines + 4 future status bits).

In our case, 10 inputs imply 1024 possible lines: complex.

!! separate control and future status to obtain 2 more easily manageable ROMs.

The opcode does not rely on the value of the controls. Hence, if we were to use a single ROM, we would have the same output combinations repeated 64 times (duplication and waste).

2_ROMs implementation – next state (ROM2)

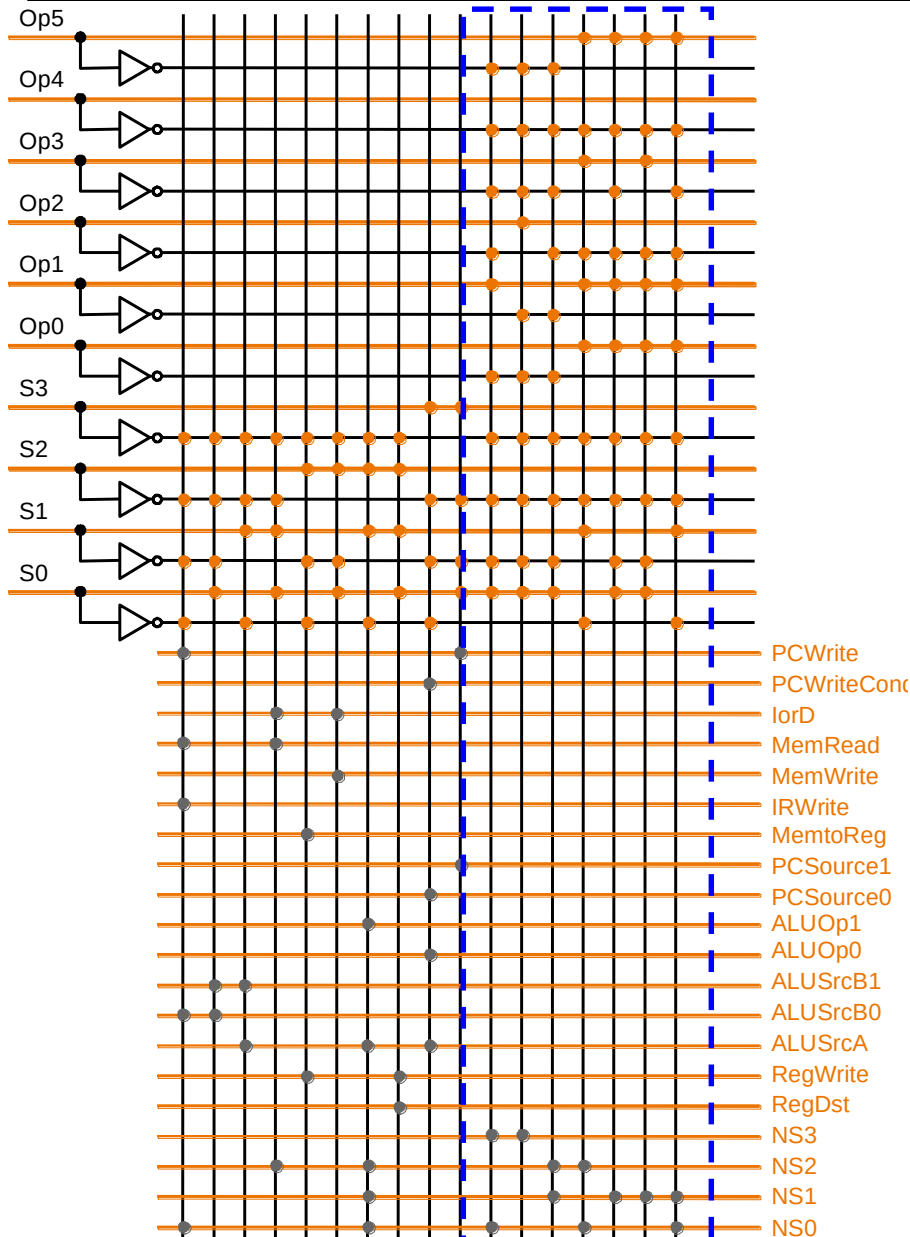
Stato corrente/ Opcode	ADD 000000	Jump 000010	Branch 000100	LW	SW	Altri valori
0000	0001	0001	0001	0001	0001	0001
0001	0110	1001	1000	0010	0010	ILLEG.
0010	xxxx	xxxx	xxxx	0011	0101	ILLEG.
0011	0100	0100	0100	0100	0100	ILLEG.
0100	0000	0000	0000	0000	0000	ILLEG.
0101	0000	0000	0000	0000	0000	ILLEG.
0110	0111	0111	0111	0111	0111	ILLEG.
0111	0000	0000	0000	0000	0000	ILLEG.
1000	0000	0000	0000	0000	0000	ILLEG.
1001	0000	0000	0000	0000	0000	ILLEG.

**L'ultima colonna
corrisponde a codici
operativi non permessi**

**Con 2 ROM risparmio:
 $2^4 \times 16 + 2^{10} \times 4 = 4.3$ Kbit
anziché 20 Kbit**

**No ottimizzazione: codici
don't care $\Rightarrow$ ancora
spreco (< di prima però).**

Control unit: PLA implementation



Less memory but a more complex address decoding circuit.

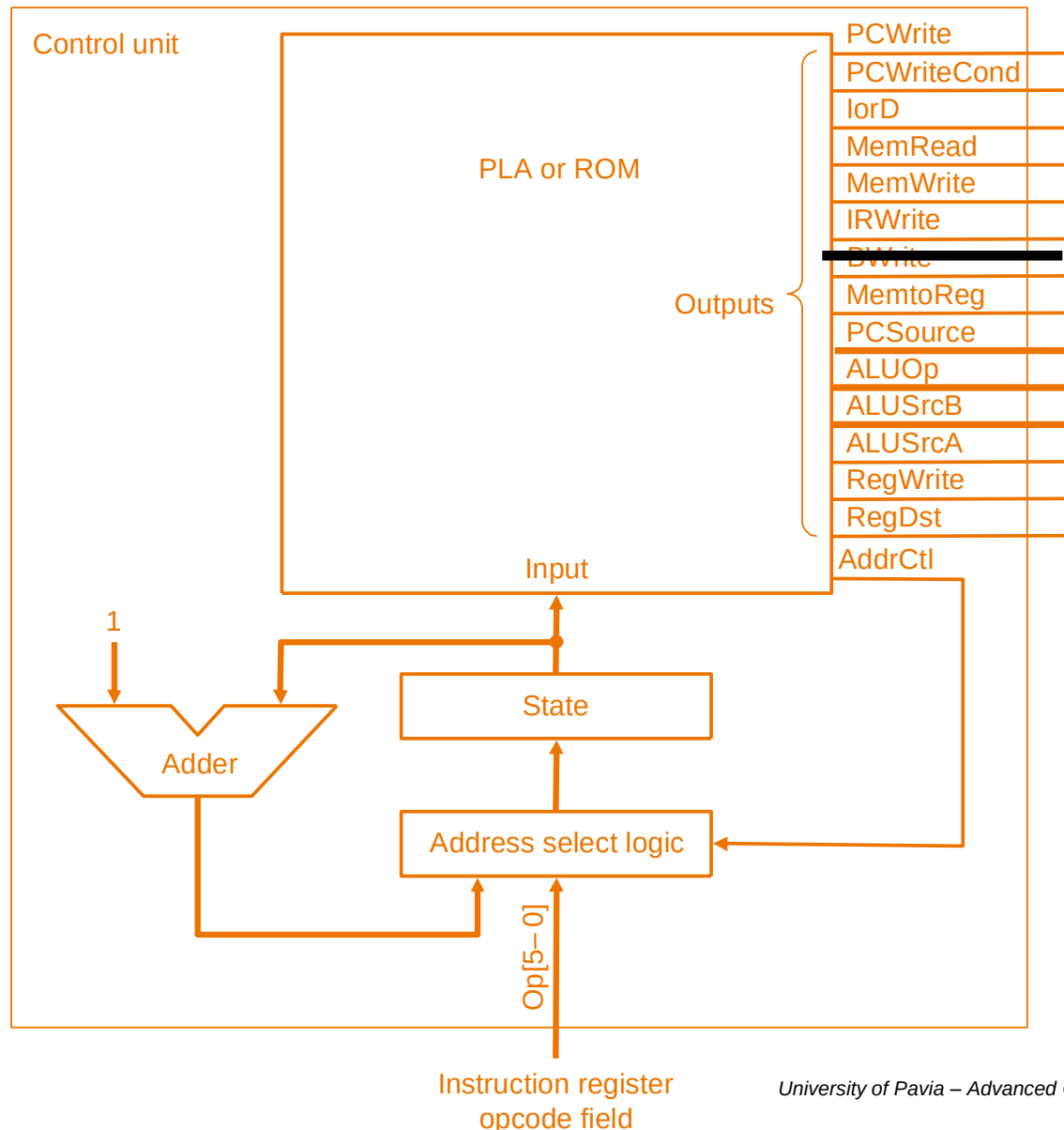
Each output is the logical OR (dark dot) of several minterms (vertical columns).

17 minterms, of which 10 depend on the current state and the other 7 depend on both the current state and the opcode.

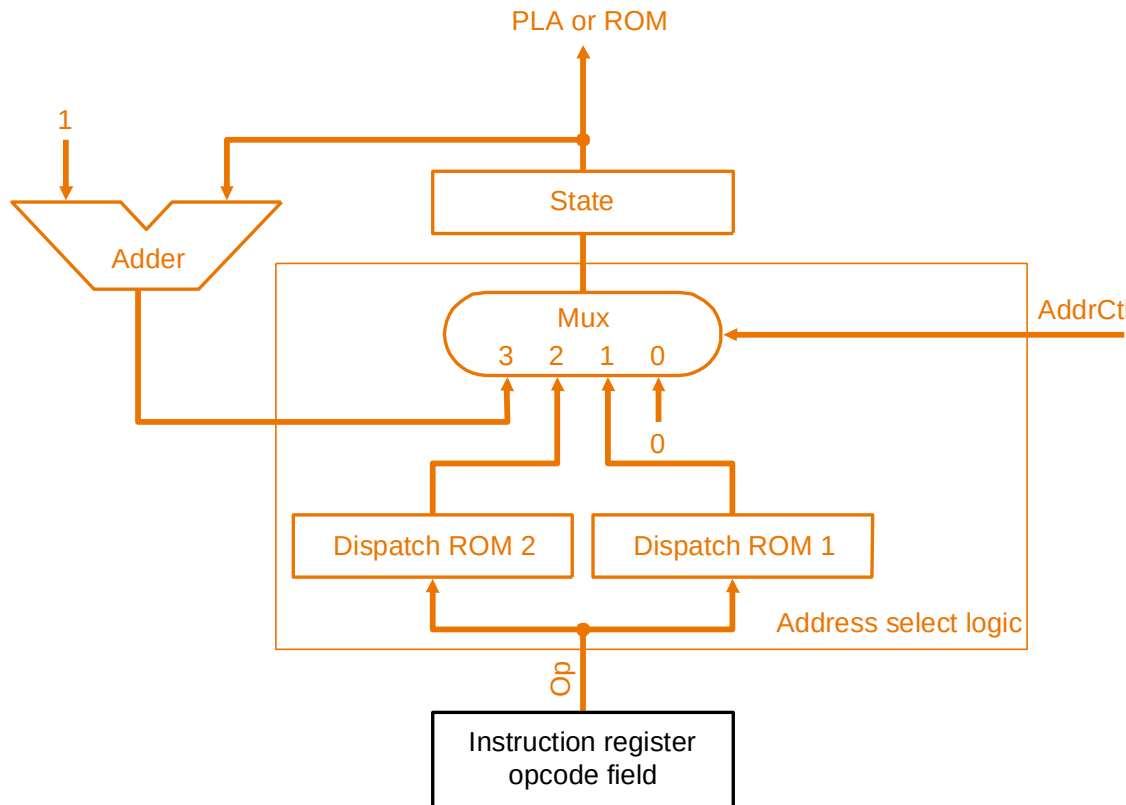
Each minterm is the AND of the corresponding terms (light dots). Size = AND plane + OR plane = $(10 \times 17) + (20 \times 17) = 510$ bits.

It could be split into two, further minimising the design: 10 inputs with 7 minterms and 4 future state outputs + 4 inputs and 10 minterms to generate the control outputs.

Next state with sequencer - 1



Next state with sequencer - 2



BRANCHES MANAGEMENT

If the next state is defined through a choice among many possibilities (as in state 1 and 2) it will be provided by a suitable external ROM/PLA Dispatch memory addressed through suitable control lines AddrCtl).

If the next state is 0, a suitable input to the multiplexer is explicitly coded.

Next state with sequencer – 3: Details

